PLATEFORME DE DONNÉES GÉOSPATIALES : PREUVE DE CONCEPT

Multi-Source Geospatial Data Platform

Fiche technique, support de préparation de réunion

13 août 2026

En bref

Ce projet est un **POC de plateforme de données géospatiales** qui unifie, sous un seul socle technique, l'ingestion de sources hétérogènes (API, fichiers, imagerie satellite), leur stockage dans un véritable *lakehouse*, et leur restitution auprès de deux publics différents : l'analyste (via un portail d'exploration et d'analyse) et l'utilisateur cartographique (via un explorateur de carte). L'ensemble tient dans un seul `docker compose up` et démarre sans aucun identifiant externe.

Sommaire

1. Contexte et problématique 2. La solution 3. Architecture technique 4. Cycle de vie d'une donnée 5. Comment le projet a été construit 6. Organisation du dépôt 7. Stack technique 8. Limites assumées du POC 9. Questions & réponses

1 Contexte et problématique

Les données utiles à la décision (démographie, occupation du sol, climat, agriculture, hydrologie) existent, mais elles sont **dispersées** entre des formats et des canaux qui ne se parlent pas : fichiers plats partagés par e-mail, API publiques à l'accès hétérogène, imagerie satellite accessible seulement via des outils spécialisés (Google Earth Engine). Chaque nouvelle source de données appelle un pipeline sur mesure, et les cartes de restitution finissent, très souvent, par être maintenues *à la main*, ce qui les rend lentes à mettre à jour et sujettes à erreur dès que la donnée source change.

La conséquence concrète : un analyste métier, non technique, ne peut ni trouver, ni croiser, ni visualiser ces données sans faire appel à un développeur à chaque étape. Le savoir reste bloqué dans des silos techniques.

> **Le POC répond à une question précise :** peut-on construire, sur une seule machine et sans infrastructure lourde, un socle qui absorbe des sources hétérogènes sous un contrat unique, tout en offrant une expérience de découverte en libre-service à quelqu'un qui n'écrit pas de SQL ?

2 La solution

La plateforme repose sur trois piliers.

2.1 Un contrat d'ingestion unique

Quelle que soit la source (API, fichier déposé, extraction satellite), chaque jeu de données est décrit par un **manifeste YAML déclaratif** : origine, contrôles qualité, gouvernance, configuration de publication. Le moteur d'orchestration lit une intention, pas une implémentation ad hoc. Ajouter une source ne veut pas dire écrire un nouveau pipeline from scratch, mais déclarer un nouveau manifeste dans un cadre déjà éprouvé.

2.2 Un vrai lakehouse

Les données atterrissent dans des tables **Apache Iceberg** (transactions ACID, schéma versionné) posées sur un stockage objet compatible S3. Le raster suit un chemin dédié (COG et catalogue STAC), le vecteur pré-rendu un autre (PMTiles). Trois catalogues distincts organisent la découverte : REST Iceberg, STAC spatio-temporel, et une table de lignage qui trace chaque exécution de pipeline.

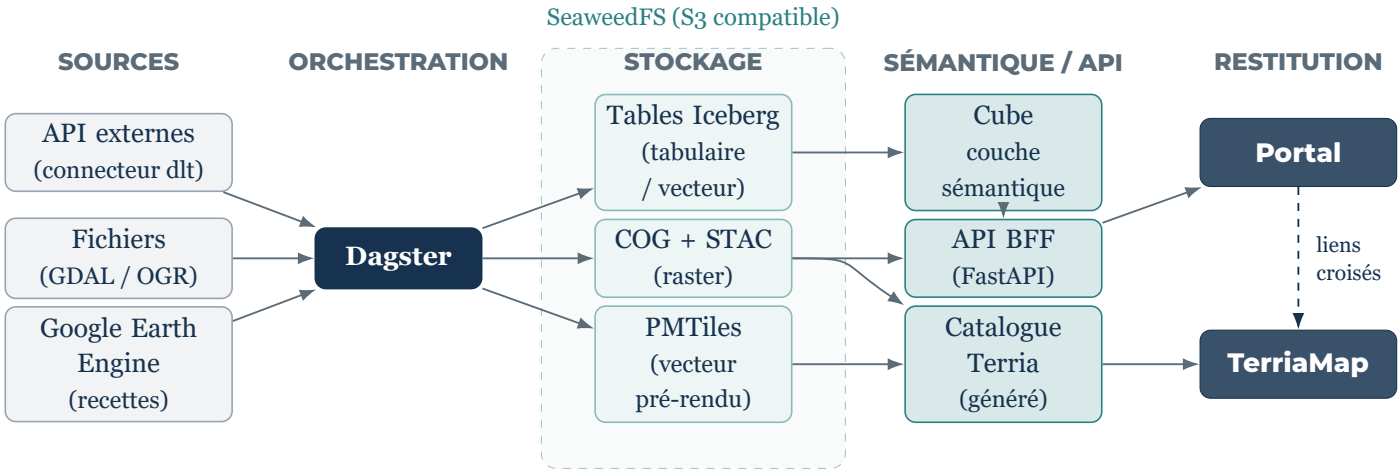
2.3 Deux interfaces, un seul public visé au global

Portal répond à la question *« quelle est la valeur de cet indicateur, et comment évolue-t-il ? »* : recherche, filtrage, graphiques, export, sans SQL. **TerriaMap** répond à *« à quoi ressemblent*

ces couches superposées sur une carte ? » : catalogue **généré automatiquement**, jamais écrit à la main. Des liens croisés font passer l'utilisateur de l'un à l'autre selon son besoin du moment.

3 Architecture technique

Le schéma ci-dessous suit le trajet d'une donnée, de la source jusqu'à l'écran de l'utilisateur final.



Trois catalogues séparés assurent la découverte : **Lakekeeper** (catalogue REST Iceberg), **pgstac / STAC API** (découverte spatio-temporelle du raster et du vecteur), et une table `platform.pipeline_runs` (lignage des exécutions). Le format de stockage suit la modalité de la donnée : Iceberg pour le tabulaire et le vecteur analytique, COG+STAC pour le raster, PMTiles pour le vecteur pré-rendu destiné à la carte.

4 Cycle de vie d'une donnée

Pour rendre l'architecture concrète, voici ce qui se passe, étape par étape, quand un nouveau jeu de données entre dans la plateforme.

1. **Déclaration** : un manifeste YAML décrit la source, son type (API, fichier, GEE), les contrôles qualité attendus et les règles de gouvernance et de publication.
2. **Exécution** : Dagster matérialise l'asset correspondant, le connecteur adapté (dlt, GDAL/OGR, ou interpréteur de recette GEE) va chercher la donnée.
3. **Normalisation** : la donnée est convertie dans le format cible selon sa modalité (GeoParquet vers Iceberg, COG pour le raster, PMTiles pour le vecteur pré-rendu via `tippecanoe`).

4. **Contrôle qualité** : des vérifications (Soda Core) s'exécutent comme *asset checks* Dagster ; un jeu de données qui échoue à un contrôle ne passe pas silencieusement.
5. **Traçabilité** : chaque exécution produit obligatoirement cinq sorties : artefact brut avec somme de contrôle, asset normalisé, enregistrement au catalogue, entrée de lignage, rapport de qualité. En produire moins est considéré comme un échec par le framework lui-même, pas par convention.
6. **Publication** : le jeu de données apparaît dans le catalogue STAC et/ou le registre de couches publiées, et le catalogue TerriaMap est régénéré automatiquement pour le refléter.
7. **Découverte** : l'utilisateur final trouve la donnée dans Portal (recherche, fiche descriptive, lignage) ou directement sur la carte dans TerriaMap, avec un lien vers l'autre interface.

5 Comment le projet a été construit

Le projet a été bâti **milestone par milestone**, chacun validé par une porte de vérification (*stop gate*) avant de passer au suivant : une commande reproductible, un résultat observable, pas d'étape « ça devrait marcher ».

Étape	Nom	Ce qui a été livré
M1	Socle stockage	SeaweedFS + Postgres, squelette Docker Compose, suite de tests de compatibilité S3
M2	Catalogue Iceberg	Lakekeeper, premier asset Dagster écrivant une table Iceberg, Jupyter + DuckDB
M3	Framework d'ingestion	Contrat de manifeste, connecteurs API et fichiers, deux jeux de données réels
M4	Raster et satellite	Pipeline raster, catalogue STAC, recette Google Earth Engine
M5	Couche sémantique	Cube, modèle métier, API BFF
M6	Portail	Interface Portal : découverte, fiche dataset, atelier d'analyse
M7	Carte	TerriaMap, catalogue généré, correspondance de régions
M8	Finition	Liens croisés entre les deux interfaces, script de démonstration, polish

Chaque décision technique non triviale, en particulier celles qui allaient à l'encontre d'une hypothèse initiale du cahier des charges, a été consignée dans un **ADR** (*Architecture Decision Record*) sous `docs/adr/` : le contexte, ce qui a été vérifié, la décision retenue et pourquoi. Six ADR documentent aujourd'hui les écarts entre le plan initial et la réalité de l'implémentation (comportement de SeaweedFS, configuration de Lakekeeper, limites de TerriaJS, etc.), et une partie d'entre eux est reprise dans la section Questions & réponses.

6 Organisation du dépôt

Le code suit une disposition pensée pour ressembler, dès le POC, à la disposition d'un projet de production : rien n'est jeté au moment de passer à l'échelle.

Dossier	Contenu
`orchestration/`	Dagster : définitions, resources, assets par jeu de données, l'asset `terria_catalog`
`connectors/`	Connecteurs par type de source : `api_source`, `file_source`, `gee_source`, socle commun
`recipes/`	Manifestes YAML des jeux de données et recettes d'extraction GEE
`semantic/`	Modèle Cube (`cubes/` et `views/`)
`apps/api`	Backend-for-frontend FastAPI
`apps/portal`	Frontend Portal (React + Vite)
`apps/terriamap`	Configuration et assets de l'explorateur cartographique
`infra/`	Configuration bas niveau (SeaweedFS, Postgres, Keycloak, Jupyter)
`sample-data/`	Données synthétiques et fixtures : fonctionnement garanti sans identifiant externe
`docs/adr/`	Une décision d'architecture documentée par fichier

7 Stack technique

Couche	Composants	Rôle
Stockage objet	SeaweedFS (compatible S3)	Buckets `raw`, `warehouse`, `assets`, `serving`
Catalogues	Lakekeeper, pgstac / STAC API, Postgres	Catalogue REST Iceberg, découverte spatio-temporelle, lignage des exécutions
Orchestration	Dagster	Pipelines déclaratifs, assets, capteurs
Couche sémantique	Cube (moteur DuckDB)	Traduction des tables physiques en métriques métier
Backend	FastAPI (BFF)	Recherche, métadonnées de jeux de données, exports
Frontends	Portal (React / Vite), TerriaMap	Analyse en libre-service, exploration cartographique
Identité	Keycloak (OIDC)	Un realm de développement unique
Notebooks	Jupyter + DuckDB	Exploration ad hoc des tables

8 Limites assumées du POC

Le projet est explicite sur ce qu'il ne prouve pas : une liste de renoncements **délibérés**, pas des oublis.

- Pas de haute disponibilité, pas de test de charge.
- Authentification limitée à un unique realm Keycloak de développement.
- Pas de stratégie de sauvegarde ou de reprise après sinistre.
- Moteur de calcul, moteur de recherche et finition d'interface ont pris des raccourcis de POC **délibérés et réversibles**.

En miroir, les décisions qui *ne* sont *pas* des raccourcis (stockage, format de table, contrat de manifeste) ont été traitées avec la même rigueur qu'en production, précisément parce qu'elles seraient coûteuses à changer plus tard. C'est la distinction centrale à défendre en réunion si la question de la maturité du projet est posée.

9 Questions & réponses : préparation réunion

9.1 Questions de base

Que fait concrètement cette plateforme ?

Elle ingère des données géospatiales et statistiques de sources variées (API, fichiers, imagerie satellite), les stocke dans un lakehouse structuré, et les rend explorables via deux interfaces web : un portail d'analyse (Portal) et un explorateur cartographique (TerriaMap).

Pourquoi deux interfaces plutôt qu'une seule ?

Parce qu'elles répondent à deux besoins différents et non redondants. Le Portal répond à *« quelle est la valeur de cet indicateur ? »* (recherche et analyse) ; TerriaMap répond à *« à quoi ça ressemble superposé sur une carte ? »* (exploration spatiale). Elles sont reliées par des liens croisés dans les deux sens.

Comment démarre-t-on la plateforme ?

Une seule commande, `make bootstrap`, lance tous les services via Docker Compose sur une seule machine, sans aucune dépendance cloud externe pour démarrer.

Qu'est-ce qu'un « lakehouse », et pourquoi est-ce préférable à un simple entrepôt de fichiers ?

C'est un stockage objet (fichiers Parquet) doté de garanties habituellement réservées aux bases de données : transactions ACID, schéma versionné, requêtes SQL, via Apache Iceberg. On garde la flexibilité et le coût d'un stockage objet tout en gagnant la fiabilité d'un entrepôt de données classique.

D'où viennent les données ?

De trois familles de connecteurs : API externes (via `dlt`), fichiers déposés (via GDAL/OGR pour le géospatial), et Google Earth Engine (recettes d'extraction satellite). Des données synthétiques et des fixtures enregistrées permettent de faire tourner la démonstration sans identifiants réels.

9.2 Questions intermédiaires

Pourquoi SeaweedFS plutôt que MinIO, plus répandu ?

C'était une contrainte du cahier des charges, justement pour valider la compatibilité S3 d'un composant moins éprouvé que MinIO. Des écarts réels ont été trouvés et documentés, par exemple des échecs intermittents sur des écritures conditionnelles, et gérés par une suite de tests de compatibilité dédiée.

Comment le catalogue de TerriaMap reste-t-il à jour ?

Il n'est jamais écrit à la main. Un asset Dagster le régénère automatiquement à partir du catalogue STAC, du registre des couches PMTiles publiées, et des couches externes. Publier un nouveau jeu de données met donc la carte à jour sans intervention manuelle.

Que se passe-t-il en l'absence d'identifiants Google Earth Engine ?

Le pipeline bascule automatiquement sur des fixtures enregistrées, et l'interface l'indique explicitement : la démonstration reste fonctionnelle et honnête sur l'origine réelle des données affichées.

Quel est le rôle de Cube dans l'architecture ?

C'est la couche sémantique : elle traduit les tables physiques Iceberg en métriques compréhensibles pour un métier (par exemple « taux d'emploi par région ») que le Portal interroge directement, sans que l'utilisateur final n'écrive de SQL.

9.3 Questions plus pointues

Pourquoi ne pas avoir utilisé Spark ou Trino, standards du marché ?

C'est un choix de POC délibéré et documenté : le volume de données en jeu ne justifie pas ce coût opérationnel. DuckDB remplace Trino pour l'exécution analytique, la recherche plein texte de Postgres remplace OpenSearch, et une simple table de lignage remplace des services dédiés comme Marquez. Ce sont des raccourcis de **calcul et de recherche**, jugés réversibles, à l'inverse des choix de stockage et de format, jugés irréversibles et donc traités avec la même rigueur qu'en production dès le POC.

Le format PMTiles est-il vraiment utilisé dans TerriaMap ?

Le pipeline génère bien du PMTiles via `tippecanoe`. Mais la version de TerriaJS utilisée ne supporte pas nativement ce type en entrée de catalogue : les couches vecteur y sont donc exposées en GeoJSON. C'est une limite de l'outil front-end au moment de la restitution, pas un renoncement à PMTiles côté stockage.

TerriaMap tourne-t-il en 3D par défaut ?

Non, il a d'abord été forcé en mode 2D. Le mode 3D par défaut nécessite un token payant pour le terrain et l'imagerie de fond ; comme le POC doit démarrer sans aucun identifiant externe, ce mode produirait un rendu inutilisable. Ce point a depuis évolué (une vue 3D inclinée a été introduite pour certains rendus) et vaut la peine d'être vérifié dans l'état courant du dépôt avant d'en parler en réunion.

Si l'exécution d'un pipeline échoue à mi-chemin, que se passe-t-il aux données ?

Le framework impose que chaque exécution produise cinq sorties obligatoires : artefact brut

avec somme de contrôle, asset normalisé, enregistrement au catalogue, entrée de lignage, rapport de qualité. Une exécution qui en produit moins est traitée comme un échec par le framework lui-même : ce n'est pas laissé à la discipline de chacun. Cela évite les états partiels silencieux dans le lakehouse.

Comment la géométrie est-elle gérée dans Iceberg, alors qu'un support géographique natif existe désormais ?

Volontairement non utilisé pour ce POC : la géométrie est stockée via un pont explicite, une colonne binaire WKB, des colonnes d'enveloppe (bbox), et le système de coordonnées déclaré dans les propriétés de la table. C'est plus verbeux, mais stable et interrogeable dès aujourd'hui, sans dépendre d'un support natif encore jeune dans l'écosystème.

Cette plateforme est-elle prête pour la production ?

Non, explicitement pas : pas de haute disponibilité, pas de test de charge, authentification limitée à un realm de développement, pas de stratégie de sauvegarde. En revanche, les décisions structurantes (stockage objet, format de table, contrat de manifeste) ont été prises pour être *migration-safe*. L'objectif du POC n'est pas d'éviter ces sujets, mais de ne pas avoir à les refaire en passant en production.

Sources : `README.md`, `documentation/MASTER_PROMPT_GEOPLATFORM_POC_v2.md`, `docs/DEMO_SCRIPT.md`, `docs/adr/0001` à `0006`.